

Warsaw University of Technology

FACULTY OF
ELECTRONICS AND INFORMATION TECHNOLOGY



Institute of Control and Computation Engineering

Bachelor's diploma thesis

in the field of study Computer Science
and specialisation Software Engineering

Web/desktop application (TypeScript) for visualizing and composing
dependency graph between reusable software components

Binh Vuong Le Duc

student record book number 330097

thesis supervisor

mgr inż. Sławomir Niespodziany

WARSAW 2026

Wersja w języku polsko-angielskim

Politechnika Warszawska
Warsaw University of Technology

.....
miejscowość i data
place and date

.....
imię i nazwisko studenta
name and surname of the student

.....
numer albumu
student record book number

.....
wydział i kierunek studiów
faculty and field of study

OŚWIADCZENIE **DECLARATION**

Świadomy/-a odpowiedzialności karnej za składanie fałszywych zeznań oświadczam, że niniejsza praca dyplomowa została napisana przeze mnie samodzielnie, pod opieką kierującego pracą dyplomową.

Under the penalty of perjury, I hereby certify that I wrote my diploma thesis on my own, under the guidance of the thesis supervisor.

Jednocześnie oświadczam, że:

I also declare that:

- niniejsza praca dyplomowa nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (Dz.U. z 2021 r., poz. 1062) oraz dóbr osobistych chronionych prawem cywilnym,
this diploma thesis does not constitute infringement of copyright following the act of 4 February 1994 on copyright and related rights (Journal of Acts of 2021, item 1062) or personal rights protected under the civil law
- niniejsza praca dyplomowa nie zawiera danych i informacji, które uzyskałem/-am w sposób niedozwolony,
the diploma thesis does not contain data or information acquired in an illegal way,
- niniejsza praca dyplomowa nie była wcześniej podstawą żadnej innej urzędowej procedury związanej z nadawaniem dyplomów lub tytułów zawodowych,
the diploma thesis has never been the basis of any other official proceedings leading to the award of diplomas or professional degrees,
- wszystkie informacje umieszczone w niniejszej pracy, uzyskane ze źródeł pisanych i elektronicznych, zostały udokumentowane w wykazie literatury odpowiednimi odnośnikami,
all information included in the diploma thesis, derived from printed and electronic sources, has been documented with relevant references in the literature section,
- znam regulacje prawne Politechniki Warszawskiej w sprawie zarządzania prawami autorskimi i prawami pokrewnymi, prawami własności przemysłowej oraz zasadami komercjalizacji.
I am aware of the regulations at Warsaw University of Technology on management of copyright and related rights, industrial property rights and commercialisation.

.....
(podpis studenta)
(signature of the student)

Web/desktop application (TypeScript) for visualizing and composing dependency graph between reusable software components

Abstract. This thesis presents the design and implementation of a desktop application that visualizes and edits dependency graphs between reusable C++ embedded software modules. Module metadata is read from an Artifactory server or local git repositories and surfaced as a draggable catalog; the user composes the dependency graph according to project needs. The output is a topology file consumed by an existing CI/CD pipeline that builds a binary tailored to the chosen module composition. The application is implemented with Electron, React, and TypeScript, and replaces the manual JSON authoring workflow identified as future work in the framework's reference paper.

Keywords: dependency injection, visual programming, embedded systems, graph editing, Electron

Aplikacja web/desktop (TypeScript) do wizualizacji i edycji grafu zależności pomiędzy reużywalnymi modułami oprogramowania

Streszczenie. Niniejsza praca przedstawia projekt i implementację aplikacji desktopowej do wizualizacji i edycji grafów reprezentujących zależności pomiędzy reużywalnymi modułami oprogramowania wbudowanego napisanymi w C++. Aplikacja odczytuje metadane modułów z serwera Artifactory lub lokalnych repozytoriów git i prezentuje je w formie przeciągalnego katalogu; użytkownik komponuje graf zależności zgodnie z potrzebami projektu. Wynikiem działania są pliki topologii wykorzystywane przez istniejący potok CI/CD do zbudowania binarnej wersji aplikacji odpowiadającej wybranemu zestawowi modułów. Aplikację zaimplementowano w technologiach Electron, React oraz TypeScript; zastępuje ona ręczne tworzenie pliku topologii w formacie JSON, wskazane jako dalszy kierunek prac w publikacji opisującej framework.

Słowa kluczowe: wstrzykiwanie zależności, programowanie wizualne, systemy wbudowane, edycja grafów, Electron

Contents

1. Introduction 11

1.1. Motivation 11

1.2. Problem Statement 11

1.3. Project Goals and Scope 11

1.4. Thesis Structure 12

References 13

List of Symbols and Abbreviations 14

List of Figures 14

List of Tables 14

List of Appendices 14

1. Introduction

1.1. Motivation

The rapid advancement of semiconductor technology and the increasing demand for sophisticated functionality in embedded systems have shifted the primary challenge of development from hardware constraints to software complexity [1]. As embedded devices integrate into critical infrastructure, such as automotive control units and industrial IoT gateways, the need for scalable and maintainable software architectures has never been greater.

In traditional embedded development, software components often exhibit coupling with the underlying hardware, hindering reusability across different projects. To address this, modular frameworks like `diff` utilize the Dependency Injection (DI) pattern [2] to decouple component logic from system configuration. While this approach improves testability and flexibility, it introduces a new layer of complexity: the management of the dependency graph itself.

In the current workflow, developers must manually define the system topology in structured JSON files. While functional, this “blind wiring” process is inherently error-prone. As system complexity increases, detecting circular dependencies, identifying unfulfilled requirements, or understanding the architecture from a text-based file becomes a significant cognitive burden. A specialized tool that provides visual feedback and automated verification during the system composition phase is essential.

1.2. Problem Statement

The core problem addressed by this thesis is the lack of a visual management layer for the `diff` dependency injection framework. Specifically:

- **High Cognitive Load:** Managing hundreds of component connections in a JSON format is difficult to audit and visualize.
- **Delayed Error Detection:** Structural issues, such as dependency cycles, are often only caught at runtime, leading to expensive debugging cycles.
- **Configuration Fragility:** Manually editing IDs and dependency arrays is susceptible to typos and reference errors that a standard text editor cannot verify.

1.3. Project Goals and Scope

The primary goal of this project is to develop `diff-forge`, a cross-platform desktop application that enables developers to visually compose, verify, and export software topologies. The application utilizes modern web technologies including TypeScript, React [3], and Electron [4]. The specific goals of the application include:

- Providing an intuitive drag-and-drop interface for instantiating components from a managed catalog.
- Implementing a real-time validation engine to prevent structural errors before they reach the production environment.

- Ensuring seamless integration with existing C++ development workflows by exporting framework-compliant configuration files.

1.4. Thesis Structure

The following chapters comprise this thesis: Chapter 2 reviews the state of the art in dependency injection and visual programming. Chapter 3 provides an overview of the `diff` framework and its target use cases. Chapter 4 describes the architectural design of `diff-forge`, while Chapter 5 details its implementation. Lastly, Chapter 6 evaluates the tool's effectiveness, followed by concluding remarks in Chapter 7.

References

- [1] S. Niespodziany, “Data-driven dependency injection for embedded software, enhancing reusability with c++”, *International Journal of Electronics and Telecommunications*, vol. 71, no. 4, pp. 1–7, 2025. DOI: 10.24425/ijet.2025.155469
- [2] M. Fowler, *Inversion of Control Containers and the Dependency Injection pattern*. martinowler.com, 2004. [Online]. Available: <https://martinfowler.com/articles/injection.html>
- [3] I. Meta Platforms. “React documentation”. [Online]. Available: <https://react.dev/>
- [4] O. Foundation. “Electron documentation”. [Online]. Available: <https://www.electronjs.org/docs/latest>

List of Symbols and Abbreviations

EiTI – Faculty of Electronics and Information Technology

WUT – Warsaw University of Technology

DI – Dependency Injection

IAiIS – Institute of Control and Computation Engineering

CBSE – Component-Based Software Engineering

GUI – Graphical User Interface

List of Figures

List of Tables

List of Appendices

1. User Manual	15
2. Configuration Schema	16

Appendix 1. User Manual

Instructions for installing and running `diff-forge`.

Appendix 2. Configuration Schema

The JSON schema used for component metadata.